



Farouk.ouatike@gmail.com

2025/2026

PLAN DU COURS

- Introduction aux bases de la programmation Python
- Structures de contrôle (boucles, conditions) et fonctions
- Manipulations des types de données
- Gestion des fichiers et des exceptions
- Programmation orienté objets en Python
- Introductions aux modules et bibliothèques courantes

Chapitre 1 :

Introduction aux bases de la programmation Python

Les langages de programmation

- Le langage de programmation est l'intermédiaire entre l'humain (anglais) et la machine (binaire).
- Il existe des milliers de langages de niveau élevé, pour tous les goûts et toutes les applications.
- Quelques uns des plus connus: C, C++, Java, PHP,...

Les langages de programmation

- Haut niveau : proche de l'homme, plus facile à apprendre et à utiliser grâce à son haut niveau d'abstraction qui masque les détails techniques du système
 - C++, Java, Python, PHP,...
- Bas niveau : proche de la machine offrant plus de contrôle et d'efficacité mais nécessitant une grande connaissance de l'architecture matérielle.
 - Assembleur

Compilation et interpréteur

- Compilation: permet de traduire le code source du programme vers le langage natif (objet) de la machine(ou parfois vers du code intermédiaire).

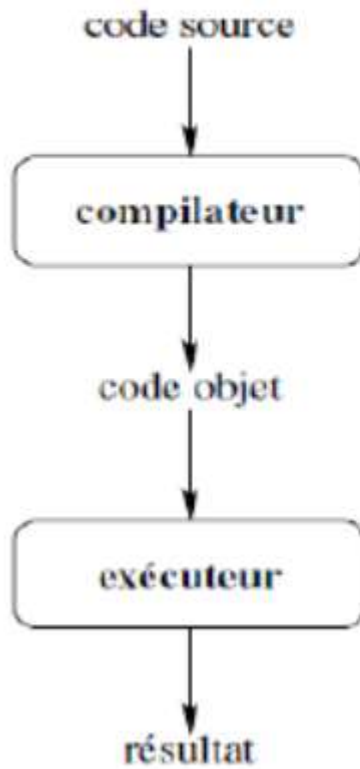


Compilation et interpréteur

Interpréteur: permet de traduire et d'exécuter chaque instruction du programme. Ce mécanisme est utilisé pour le passage d'un programme précompilé à un pseudo-code (cas de Java).

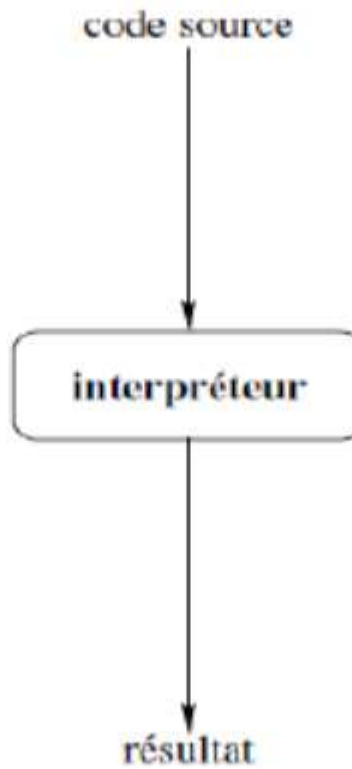
compilation

ADA, C/C++, Fortran



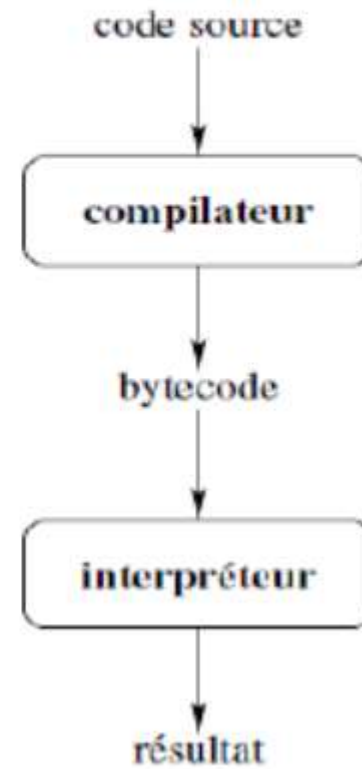
interprétation

Lisp, Prolog



semi-compilation

Java, Python



Programme et langage de programmation

- Un **programme** est un algorithme écrit dans un langage de programmation.
- Un **langage** est un ensemble de phrases.

Exemples de phrases :

- en français (« Je m'appelle Paul. ») ;
- en anglais (« The book is on the table ! ») ;
- en arithmétique (« $1 + 2 = 3$ ») ;

Programme et langage de programmation

- Un **langage de programmation** est un langage qui sert à codifier des algorithmes afin de les rendre « compréhensibles » par la machine.

- Exemple en Python :

```
def succ(x=0):  
    return x + 1
```

Différents langages de programmation

Il existe plusieurs langages de programmation :

- impératifs (C, Pascal, FORTRAN, . . .) ;
- fonctionnels (Lisp, Haskell, Scheme, ML, . . .) ;
- orientés objet (SmallTalk, C++, Java, . . .) ;
- déclaratif (PROLOG).

Le langage Python

- C'est un langage de scripts (de petits programmes très simples chargés d'une mission très précise sur votre ordinateur) ;
- Un langage impératif interprété (c'est-à-dire que les instructions que vous lui envoyez sont "transcrites" en langage machine au fur et à mesure de leur lecture) contrairement à un langage compilé (C/C++)

Le langage Python

Python est un langage de programmation.

Ce qui définit un langage de programmation:

- La façon de représenter symboliquement les structures de données.
- La façon de gérer le contrôle des programmes (que faire et dans quel ordre)

Pourquoi python

- Gratuit
- Syntaxe simple
- Portable (Mac OS, Unix, Windows, Android, etc.)
- Compilé et interprété avec une machine virtuelle
- Moderne (dynamiquement typé, multi-threadé, extensible, etc.)
- Utilisé dans des nombreux projets industriels : Arts, business, éducation, ingénierie, gouvernement, scientifique, etc.
- En évolution

Les types de données

Comme tout langage de programmation, Python fournit un certain nombre de types de données prédéfinis (booléens, entiers, flottants, chaîne de caractères, listes) et de structure de contrôle (les boucles for, while et les conditions if then else).

Les types de données

On va se limiter à quelques types :

- **le type des nombres entiers** `int`. On peut les additionner, les multiplier, les comparer,...
- **le type des nombres à virgule** `float` 8. On peut aussi les multiplier, les comparer,... et se servir de nombres entiers comme s'ils étaient des nombres à virgule.
- **le type des chaînes de caractères** `str`. On peut les afficher, les concaténer,...
- **le type des booléens** `bool`, qui valent soit **True** (vrai), soit **False** (faux).

Les nombres : entiers et flottants

- Tous les entiers (en anglais integer) au sens mathématique peuvent être représentés en python. Les flottants (en anglais float) représentent les nombres non-entiers. Ces nombres comportent un point séparant la partie entière et la partie fractionnelle.
- Par exemple 1.5 est un flottant. Sa partie entière est 1 et sa partie fractionnelle est le 5. C'est le point qui permet à python de reconnaître un flottant. Ainsi 1.0 et 1. sont des flottants, même si leur partie fractionnelle est nulle.

Les nombres : entiers et flottants

Opérateurs

- Une opération est constituée d'un opérateur et d'un ou plusieurs opérandes.

$$\begin{array}{ccc} & \text{opérateur} & \\ 2 & + & 3 \\ \text{opérande} & & \text{opérande} \end{array}$$

Les nombres : entiers et flottants

Les opérateurs suivants opèrent aussi bien sur les opérandes entières que flottantes. Les opérateurs dits unaires admettent une seule opérande, alors que les opérateurs dits binaires en admettent deux.

Opérateur	Opération
**	puissance
+ (unaire)	nombre positif
- (unaire)	changement de signe
*	multiplication
/	division réelle (sans reste)
//	quotient de la division euclidienne
%	<i>modulo</i> ou reste de la division euclidienne
+ (binaire)	addition
- (binaire)	soustraction

Les nombres : entiers et flottants

Pour la division on a : la division réelle (/) et la division euclidienne (//)

```
>>> 17 / 7
2.4585714
>>> 17 // 7
2
>>> 17 % 7
3
>>> 17.3 // 2.5
6.0
>>> 17.3 % 2.5
2.3
```

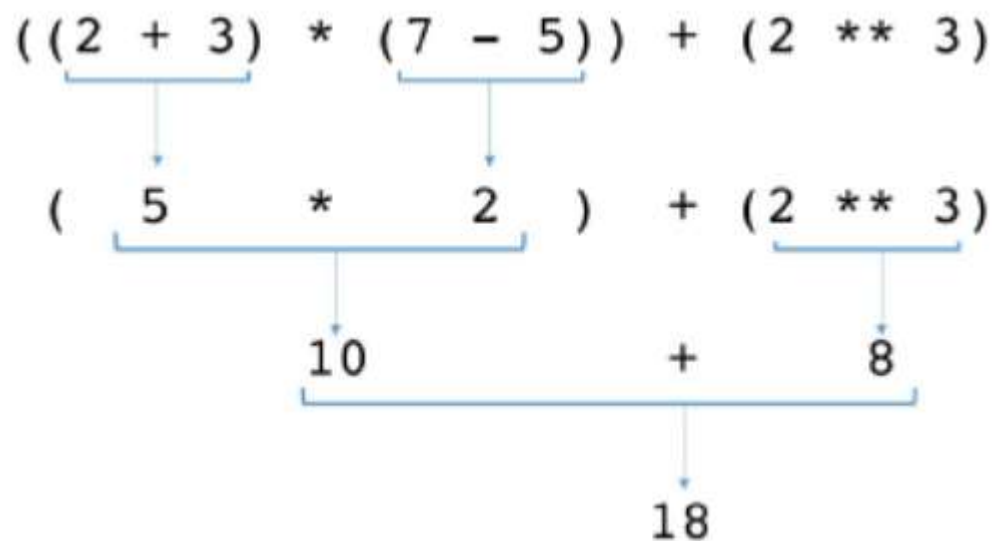
Les nombres : entiers et flottants

- L'ordre d'évaluation des opérateurs:
- Dans une expression il peut y avoir plusieurs opérateurs et, dans certains cas, l'ordre dans lequel on effectue ces opérations peut influencer sur le résultat du calcul.
- La priorité des opérateurs :

()
**
+ et - (unaires)
*, /, // et %
+ et - (binaires)

Les nombres : entiers et flottants

- L'ordre d'évaluation des opérateurs:
- Exemple : on commence par les parenthèses les plus profondes



Les nombres : entiers et flottants

Les règles :

- Commencer par effectuer les opérateurs de plus haut niveau de priorité ;
- Lorsque plusieurs opérateurs ont le même niveau de priorité, effectuer les opérations en commençant par les opérateurs les plus à gauche ...
- sauf lorsqu'il s'agit de l'opérateur puissance (où on commence par les opérateurs les plus à droite).

Les nombres : entiers et flottants

- Exemple :

The diagram illustrates the evaluation of the expression $-3 ** 2 // 2 + 3 * -7 - 5$ through a series of steps, with annotations explaining the operations performed:

- $-3 ** 2 // 2 + 3 * -7 - 5$ puissance (**)
- $-9 // 2 + 3 * -7 - 5$ signes (+, -, unaires)
- $(-9) // 2 + 3 * (-7) - 5$ multiplications, divisions (*, //)
- $(-5) + (-21) - 5$ +, - binaires (gauche)
- $(-26) - 5$ +, - binaires (droite)
- -31

Les nombres : entiers et flottants

- $-3**2//2+3*-7-5$
- $3**2 = 9$
- $-9//2 + 3*-7 - 5$
- $-9 // 2 = -5$ (car division entière en Python $\rightarrow$ arrondi vers le bas).
- $-5 + 3*-7 - 5$
- $3 * -7 = -21$
- $-5 + (-21) - 5$
- $-5 - 21 - 5 = -31$

Les chaînes de caractères:

Python permet de manipuler du texte. Les données représentant un texte sont de type chaîne de caractères (en anglais character string ou simplement string). Une chaîne de caractères est une juxtaposition de caractères. Il peut s'agir de caractères entrés au clavier :

- des lettres minuscules ou majuscules ;
- des chiffres ;
- des caractères espace, tabulation, retour à la ligne ;
- d'autres caractères visibles : @, &, #, \$, £, (, {, [etc.
- Mais les caractères peuvent aussi être invisibles (par exemple lorsque la chaîne n'a pas été entrée par l'utilisateur mais provient de la lecture d'un fichier).

Les chaînes de caractères:

Les chaînes de caractères définies de façon explicite (en anglais string literals) sont toujours encadrées par des apostrophes (en anglais quotes) ou des guillemets (en anglais double quotes) de façon équivalente. Voici des exemples de chaînes de caractères : "bonjour" ou encore 'fzlseher@lh'.

Les chaînes de caractères:

les opérateurs:

- $+$: l'opérateur de concaténation ;
- $*$: l'opérateur de duplication.

Les chaînes de caractères:

L'opérateur de concaténation `+` doit avoir deux opérandes de type chaîne de caractères. L'expression résultante est aussi une chaîne de caractères obtenue en juxtaposant les deux opérandes.

- Exemple :

```
>>> "bonjour ! " + "comment allez-vous ?"  
"bonjour ! comment allez-vous ?"
```

Les chaînes de caractères:

L'opérateur de duplication `*` doit avoir une opérande de type chaîne de caractères et une opérande de type entier. L'expression résultante est aussi une chaîne de caractères obtenue en répétant la chaîne de caractères. Le nombre de répétitions est donné par l'opérande entière.

```
>>> 3 * "Ohe "  
"Ohe Ohe Ohe "
```

Les chaînes de caractères:

- Exemple:

```
>>> 5 + 6
11
>>> "5" + "6"
"56"
>>> "numero" + "9"
"numero9"
>>> "numero" + 9
\texttt{TypeError: unsupported operand type(s) for +: 'int' and 'str'}
```

Les chaînes de caractères:

- Conversions explicites:

```
>>> str(132)
"132"
>>> str(2.5)
"2.5"
>>> int("52")
52
>>> float("5.2")
5.2
```

Les chaînes de caractères:

- Attention!

```
>>> int("n13")
File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'n13'
>>> float("1,25")
File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: '1,25'
```

Les chaînes de caractères:

- On peut aussi convertir un int à float ou l'inverse:

```
>>> float(132)
132.0
>>> int(2.5)
2
```

Les chaînes de caractères:

- **len** : la longueur d'une chaîne
- Si *s* est une chaîne de caractères, alors la valeur de `len(s)` est un entier représentant le nombre de caractères qui constituent *s*.
- Par exemple `len('coucou')` vaut 6.

Les chaînes de caractères:

- Les séquences d'échappement

Séquence	signification
\'	apostrophe texte
\"	guillemets texte
\\	backslash texte

Caractère	signification
\n	retour à la ligne
\t	tabulation
\b	retour en arrière

Les chaînes de caractères:

- Exemple :

```
>>> print('J\'ai dit non !!!')
J'ai dit non !!!
>>> print("Comment \"non\" ???")
Comment "non" ???
>>> print("\t No !!! \n \t Nein !!! \n \t Niet !!! \n \t M... !!! \n")
    No !!!
    Nein !!!
    Niet !!!
    M... !!!
```

Les variables

La plupart des langages de programmation permettent d'associer une valeur donnée à un nom.

Par exemple en python, on peut écrire :

```
Taille=8
```

Cette instruction associe la valeur 8 au nom taille . Après cette instruction, lorsque Python rencontrera le mot taille, il l'interprétera comme l'entier 8. On dit qu'une variable a été définie.

Les variables

- L'instruction qui associe une valeur à un nom est appelée une définition.
- Une définition crée une variable.
- Une variable est caractérisée par une valeur, un type et un nom (appelé aussi identificateur).
- Cet identificateur peut être constitué de lettres (minuscules ou majuscules), de chiffres et de _ (underscore). Le premier caractère ne doit pas être un chiffre.

Les variables

- Exemple:

```
>>> taille = 8
>>> aire = taille * taille
>>> print(taille)
8
>>> print(aire)
64
```

```
>>> aire = taille * taille
File "<stdin>", line 1, in <module>
NameError: name 'taille' is not defined
```

Les variables

- L'opérateur $=$, malgré sa ressemblance avec l'égalité mathématique, n'est pas symétrique. Autrement dit $a = b$ n'est pas équivalent à $b = a$.
- Le membre de gauche doit être constitué d'une variable.
- Le membre de droite peut être constitué de n'importe quelle expression composée d'une ou plusieurs constantes ou variables.
- Le membre de gauche reçoit la valeur du membre de droite

Les variables

```
>>> 8 = taille  
File "<stdin>", line 1  
SyntaxError: can't assign to literal
```

Les variables

Opérateurs:

Dans certains langages (en particulier les langages fonctionnelles), les variables gardent la même valeur pendant tout le programme (on parle, malgré tout de variables). Dans d'autres langages (en particulier en python), on peut redéfinir une variable, c'est à dire y associer une nouvelle valeur :

```
>>> a = 5
>>> a = 8
>>> print(a)
>>> 8
```

Les variables

- Opérateurs
- $a = 7.2$

opérateur	exemple	instruction équivalente	la valeur de a
+=	a += 2	a = a + 2	9.2
-=	a -= 2	a = a - 2	5.2
*=	a *= 2	a = a * 2	14.4
/=	a /= 2	a = a / 2	3.6
//=	a //= 2	a = a // 2	3.0
%=	a %= 2	a = a % 2	1.2
**=	a **= 2	a = a ** 2	51.84

Les variables

- Attention: Les variables de type chaîne de caractères n'ont pas besoin d'être mis entre guillemets.

```
>>> message = "bonjour"  
>>> message + " la compagnie"  
'bonjour la compagnie'  
>>> "message" + " la compagnie"  
'message la compagnie'
```

Les variables

Réciproquement, python essaiera d'interpréter comme une variable toute suite de caractères qui ne se trouve pas entre des apostrophes ou des guillemets. Par exemple :

```
>>> comp = " la compagnie"
>>> "bonjour" + comp
'bonjour la compagnie'
>>> bonjour + comp
File "<stdin>", line 1, in <module>
NameError: name 'bonjour' is not defined
```

Les variables

L'utilité des variables

- Stocker pour ne pas refaire le calcul lorsqu'un même calcul doit être fait à plusieurs endroits d'un programme, il vaut mieux effectuer le calcul une fois et de mettre le résultat dans une variable.
- Les valeurs littérales ne doivent apparaître qu'une seule fois (Une valeur littérale est une valeur apparaissant dans un programme sous la forme d'un chiffre. Une chaîne de caractère littérale est une chaîne entourée d'apostrophe ou de guillemets. Les données dans un programme apparaissent soit sous la forme de valeurs littérales soit sous la forme de variables.)
- La lisibilité : rendre le programme plus lisible

Les variables

- Exemple :
- Un conducteur souhaite estimer le coût total de son voyage entre deux villes aller-retour.
 - Sa voiture consomme **4,57 litres de carburant pour 100 km.**
 - La distance entre les deux villes est de **613 km.**
 - Le prix du carburant est de **1,62 dirhams par litre.**
 - Le trajet comprend également un **péage de 32,40 dirhams.**

Les variables

Exemple

Nous voulons calculer le coût du voyage en utilisant des variables.

D'abord, nous enregistrons les valeurs du problème dans des variables :

- la voiture fait 4,57 litres au 100 km.
- la distance entre les deux villes est de 613 km.
- le prix du carburant est de 1,62dh le litre.
- le péage coûtera 32,40dh.

Les variables

```
>>> conso = 4.57  
>>> dist = 613  
>>> prix_carb = 1.62  
>>> peage = 32.40
```

- Ensuite, nous calculons le coût du carburant pour un aller simple en utilisant les variables appropriées :

```
>>> cout_carb = conso * (dist / 100) * prix_carb
```

- Nous calculons le coût total du voyage aller-retour :

```
>>> total = (cout_carb + peage) * 2
```

```
>>> total  
155.56568400000003
```

Le type booléen

Une variable booléenne peut prendre deux valeurs : True (vrai) ou False (faux). Attention la majuscule est importante en python.

Le type booléen

Il existe en python d'autres opérateurs permettant de comparer deux expressions :

Opérateur	Sens
>	strictement supérieur à
<	strictement inférieur à
>=	supérieur ou égal à
<=	inférieur ou égal à
==	égal à
!=	différent de
in	appartenant à
not in	n'appartenant pas à

Attention à ne pas confondre l'opérateur "=" qui permet l'affectation d'une valeur à une variable et l'opérateur "==" qui permet la comparaison entre deux variables.

Le type booléen

Les opérations utilisant des opérateurs de comparaison retournent un booléen.

```
>>> 1.5 > 2.5  
False  
>>> 2 == 1  
False  
>>> "world" in "hello world"  
True
```

Le type booléen

On peut ainsi définir une variable booléenne comme n'importe quel autre type de variable :

```
>>> a = True  
>>> print(a)  
True
```

Le type booléen

On peut comparer des variables entre elles :

```
>>> a = 2  
>>> b = 3  
>>> a > b  
False
```

Le type booléen

On peut créer une variable contenant le résultat d'une opération de comparaison :

```
>>> variable_booleenne = (1 != 1)
>>> print(variable_booleenne)
False
```

Le type booléen

Comparaison entre nombres de types différents

En python les opérateurs de comparaison (sauf le "in" et "not in") permettent de comparer tous les nombres quels que soient leur type (entier, flottant...).

```
>>> 1.0 <= 1 # Comparaison entre flottant et entier
True
>>> 1e2 == 100 # Comparaison entre notation scientifique et entier
>>> True
```

Le type booléen

Les opérateurs "in" et "not in" permettent de tester l'appartenance ou non de nombres dans des ensembles de nombres mais cela nécessite l'utilisation de structures que nous n'avons pas encore vues dans ce cours.

Le type booléen

Comparaison entre chaînes de caractères et Unicode

- Les opérateurs de comparaison sont également définis pour comparer des chaînes de caractères (string).
- Pour pouvoir comparer des chaînes de caractères, il faut définir un ordre permettant de dire pour tout couple d'éléments lequel est supérieur ou inférieur. Cet ordre dépend de la manière dont le texte est encodé.

Le type booléen

L'encodage est le processus qui assigne à chaque caractère (par exemple "a", "A", "n", "2"...), un nombre unique appelé code point. Python 3 utilise par défaut l'Unicode pour encoder les chaînes de caractères.

Le type booléen

Le code point d'un caractère peut être affiché en utilisant la fonction `ord()` :

```
>>> ord("a")
97
>>> ord("b")
98
>>> ord("A")
65
>>> ord("/")
47
```

Le type booléen

Lorsqu'on compare deux caractères avec un opérateur de comparaison, ce sont les code points des caractères qui sont comparés.

Par exemple :

```
>>> "a" > "A" # car ord("a") > ord("A")  
True  
>>> "a" > "b" # car ord("a") < ord("b")  
False
```

Le type booléen

En pratique, il faut retenir que les lettres sont ordonnées par ordre alphabétique et les chiffres sont ordonnés par ordre croissant. De plus, l'ensemble des chiffres sont positionnés avant les lettres majuscules qui sont elles même avant les lettres minuscules :

Caractère	Code point
"1", "2", ... "9"	de 49 à 57
"A", "B", ..., "Z"	de 65 à 90
"a", "b", ..., "z"	de 97 à 122

Le type booléen

- La comparaison de chaînes de caractères se fait caractère par caractère.
- Pour les égalités "==", chaque caractère deux à deux doit être égal pour retourner True et pour les différences "!=" , il suffit d'un caractère différent entre les deux chaînes pour retourner True.

```
>>> "bonjour" == "bonjour"
True
>>> "bonjour " == "bonjour"
False
>>> "bonjour" != "bonjour!"
True
```

Le type booléen

- Pour les inégalités strictes (" $>$ ", " $<$ "), les premier caractères de chaque chaînes sont comparés, s'ils sont différents, l'expression peut être évaluée (True ou False). Sinon les deux caractères suivants sont comparés, etc.
- Si une chaîne est plus longue que l'autre et que la comparaison n'est plus possible car il manque un caractère, la chaîne la plus longue est considérée supérieure à la chaîne plus courte.

Le type booléen

- Dans le premier cas, la comparaison du "j" et "J" permet de conclure car $\text{ord}(\text{"j"}) > \text{ord}(\text{"J"})$.
- Dans le deuxième cas, la comparaison du "B" et "b" permet de conclure car $\text{ord}(\text{"B"}) < \text{ord}(\text{"b"})$.
- Dans le troisième cas, tous les caractères sont égaux deux à deux, sauf le dernier de la première chaîne qui ne peut pas être comparé. La première chaîne étant plus longue que la deuxième, elle est considérée comme supérieure.

```
>>> "bonjour" > "bonJour"  
True  
>>> "Bonjour" > "bonJour"  
False  
>>> "bonjour!" > "bonjour"  
True
```

Le type booléen

L'opérateur "in" renvoie True si chaque caractère de l'opérande gauche est présent dans le même ordre dans l'opérande droit. Inversement, l'opérateur "not in" renvoie True si la chaîne de caractère de l'opérande gauche n'est pas présente caractère par caractère dans l'opérande droit.

```
>>> "hello" in "hello !"
True
>>> "Hello" in "hello !"
False
>>> "hello !" not in "helo !"
True
```

Le type booléen

La comparaison de chaînes de caractères n'est pas forcément intuitive. Il faut bien faire attention à la constitution des chaînes de caractères que vous comparez avant de vous en servir dans vos programmes. Si elles ne contiennent que des lettres par exemple, le résultat pourrait vous surprendre.